

Informe de tecnologías seleccionadas para la aplicación móvil

Proyecto: coctelera automática con integración IoT sobre ESP32

Objetivo	Documentar el stack tecnológico de la app móvil, justificar cada tecnología elegida y relacionarla con la arquitectura observada en el diagrama de clases.
Alcance	Se cubren frontend, persistencia local, comunicación con el dispositivo, organización de capas de software y criterios de escalabilidad de la aplicación.

Fecha: 06-04-2026

Resumen ejecutivo

A partir del diagrama de clases de la aplicación móvil, se propone una arquitectura basada en capas claras: interfaz de usuario, gestión de estado, servicios de dominio, repositorios de persistencia local y adaptadores de comunicación con el dispositivo IoT. La decisión principal es utilizar React Native como base del frontend móvil, complementado con TypeScript, Expo, Expo Router y SQLite. Esta selección prioriza velocidad de desarrollo, reutilización del conocimiento actual del equipo, facilidad para integrar navegación por pestañas, soporte a operación offline y capacidad de crecer hacia una arquitectura más robusta sin rehacer la base del proyecto.

En términos prácticos, la app queda preparada para tres módulos funcionales visibles en el diseño: Control, Tragos y Ajustes. Debajo de esa capa de pantallas, el diagrama muestra componentes como AppStore, DeviceService, CommandQueueService, repositorios y adaptadores BLE/MQTT. Esa estructura permite separar la lógica de presentación de la lógica de negocio y, al mismo tiempo, mantener abierta la posibilidad de cambiar el mecanismo de comunicación con el ESP32 sin modificar las pantallas.

1. Contexto arquitectónico de la app

El diagrama de clases representa una app móvil centrada en el control de una coctelera automática. La estructura se organiza alrededor de tres pantallas principales —ControlScreen, DrinksScreen y SettingsScreen— y una navegación por pestañas. Cada pantalla delega responsabilidades en componentes más específicos: stores para el estado, services para la lógica de aplicación, repositories para persistencia local y adapters para la comunicación con el dispositivo.

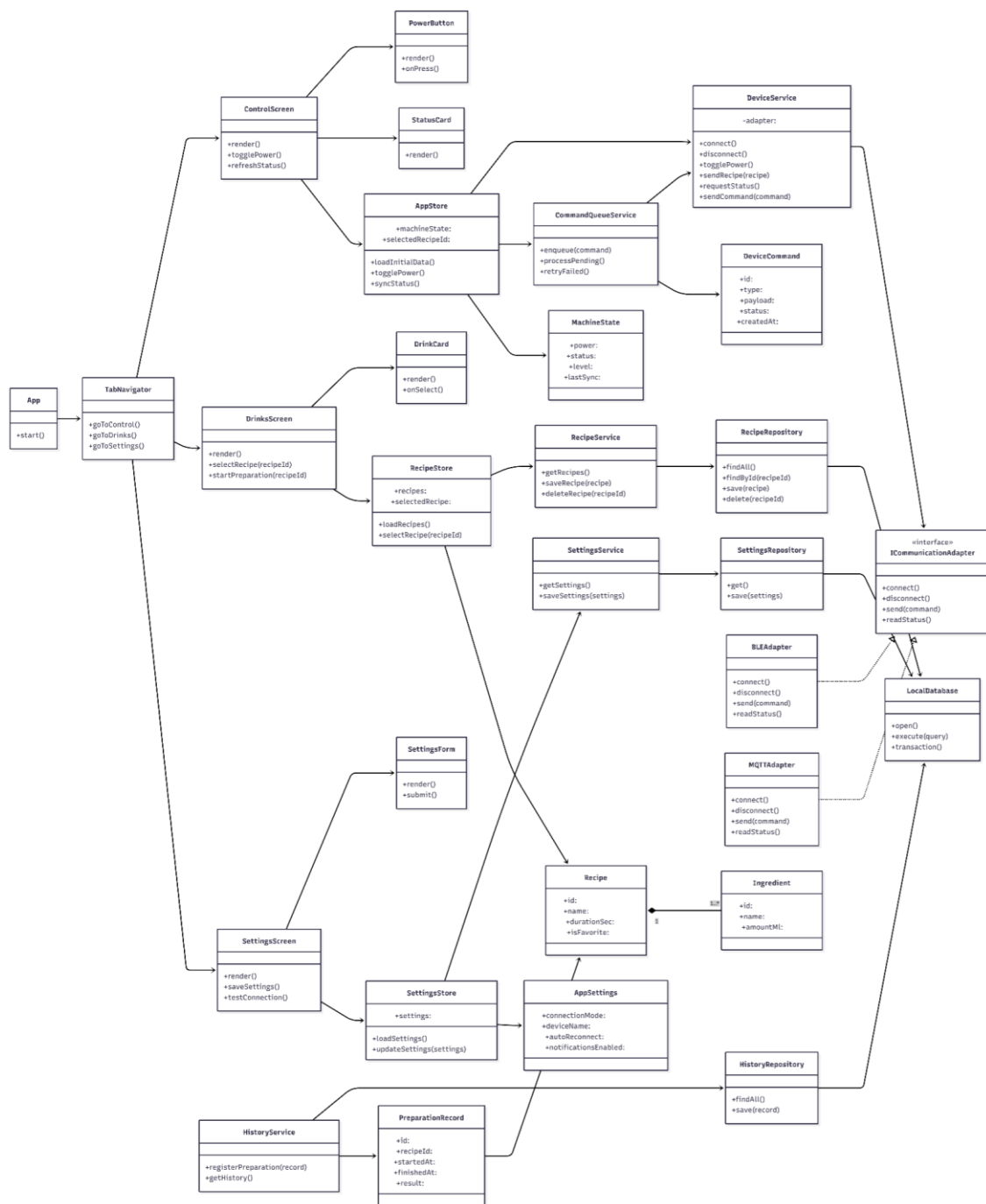


Figura 1. Diagrama de clases de la aplicación móvil utilizado como base del presente informe.

Observaciones clave del diagrama:

- La interfaz está desacoplada de la comunicación con el hardware; las pantallas no hablan directamente con el ESP32.
- La comunicación se abstrae mediante la interfaz `ICommunicationAdapter`, lo que permite trabajar con BLE en el MVP y con MQTT en una evolución futura.
- La persistencia local se concentra en `LocalDatabase` y en repositorios específicos, lo que evita mezclar acceso a datos con lógica de presentación.
- La presencia de `AppStore`, `RecipeStore` y `SettingsStore` evidencia la necesidad de manejar estado compartido y consistente entre módulos.

2. Criterios de selección tecnológica

Criterio	Aplicación al proyecto
Alineación con el equipo	Se privilegió React Native porque el equipo ya conoce React. Esta decisión reduce la curva de aprendizaje, acelera el inicio del proyecto y disminuye el riesgo de retrabajo.
Tiempo de implementación	El proyecto requiere una base funcional rápida para validar el control del equipo físico. Por ello se priorizaron tecnologías con una experiencia de arranque madura y una configuración razonable.
Modularidad	La arquitectura debía adaptarse al diagrama de clases propuesto, especialmente a la separación entre pantallas, stores, servicios, repositorios y adaptadores.
Operación local y tolerancia a desconexión	La app necesita poder mantener configuración, recetas, historial y estado reciente sin depender completamente de internet.
Escalabilidad	Aunque el primer objetivo es un MVP, la base elegida debe permitir incorporar telemetría, monitoreo remoto, sincronización y backend futuro sin reemplazar el frontend.

3. Stack tecnológico seleccionado

La siguiente tabla resume la selección principal para la app móvil y la capa de integración con el dispositivo. El foco del informe es la app; sin embargo, se incluye la tecnología de enlace con el ESP32 porque condiciona decisiones estructurales visibles en el diagrama de clases.

Capa	Tecnología	Justificación
Frontend móvil	React Native	Permite construir la app con el paradigma que el equipo ya domina y mantener una sola base de código móvil.
Lenguaje	TypeScript	Aporta tipado estático, contratos más claros entre clases y mejor mantenibilidad del código.
Framework de arranque	Expo	Simplifica la inicialización, el ciclo de desarrollo y el acceso a librerías del ecosistema.
Navegación	Expo Router	Encaja con una app modular por pantallas y tabs, y simplifica la organización del proyecto.
Persistencia local	SQLite (expo-sqlite)	Permite guardar recetas, historial, configuración y caché local de forma persistente.
Estado compartido	Stores propios	Se ajusta al diagrama mediante AppStore, RecipeStore y SettingsStore, evitando dependencia temprana de

Capa	Tecnología	Justificación
		librerías externas.
Comunicación IoT	BLE en el MVP; MQTT como evolución	BLE cubre el control cercano del dispositivo. MQTT queda preparado para escalamiento y telemetría.
Patrón de integración	Adapter + Service	ICommunicationAdapter, BLEAdapter y MQTTAdapter desacoplan la app del canal de comunicación concreto.

4. Justificación por tecnología

4.1 React Native

React Native se selecciona como base del frontend porque reutiliza el conocimiento existente del equipo en React y permite acelerar el desarrollo de una interfaz móvil multiplataforma sin duplicar esfuerzos.

Además, la estructura por componentes se adapta bien al diseño observado en el frontend y al diagrama de clases, donde existen pantallas, tarjetas, formularios y botones especializados.

También es una elección coherente con la recomendación actual de iniciar nuevos proyectos React Native con un framework como Expo, lo que reduce fricción en la puesta en marcha.

4.2 TypeScript

El diagrama expone múltiples clases con responsabilidades delimitadas, atributos y operaciones explícitas. TypeScript aporta valor directo en este escenario porque permite expresar contratos de datos y de comportamiento con mayor precisión.

En la práctica, el tipado ayuda a modelar correctamente entidades como Recipe, MachineState, DeviceCommand y AppSettings, evitando errores por propiedades faltantes, cambios no controlados o payloads mal formados.

También mejora la comunicación entre frontend y capa de servicios, especialmente cuando se implementen adapters de comunicación y repositorios con interfaces estables.

4.3 Expo

Expo se elige como framework operativo para el proyecto porque simplifica la creación del entorno inicial, la ejecución durante desarrollo, la integración con módulos del ecosistema y el mantenimiento general de la app.

Para este proyecto, Expo entrega una base práctica sobre la cual montar navegación, persistencia y pruebas en dispositivo, sin obligar al equipo a configurar desde cero toda la cadena nativa desde el primer día.

La elección es especialmente útil en una fase de MVP, donde importa avanzar rápido pero con una base ordenada.

4.4 Expo Router

La app está naturalmente organizada por pantallas y pestañas: Control, Tragos y Ajustes. Expo Router encaja bien con esta estructura porque ofrece un esquema de navegación basado en archivos, fácil de leer y mantener.

Su uso favorece una correspondencia directa entre la organización física del proyecto y la organización funcional de la aplicación, lo que ayuda a nuevos integrantes a ubicarse rápidamente en el código.

Además, permite crecer hacia flujos más complejos sin reestructurar completamente la navegación.

4.5 SQLite con expo-sqlite

La persistencia local es necesaria porque la app debe manejar recetas, historial de preparaciones, configuraciones, estado reciente y posibles comandos pendientes. SQLite resulta una opción sólida para este tipo de información estructurada y local.

La librería expo-sqlite permite integrar esta base sin salir del stack de Expo y deja los datos persistidos entre reinicios de la aplicación.

La elección también conversa bien con el diagrama, donde LocalDatabase concentra el acceso persistente y los repositorios encapsulan la lógica de lectura y escritura.

4.6 BLE para la primera etapa y MQTT como evolución

La app necesita enviar comandos a un ESP32 y consultar estado. Para un primer alcance, BLE es la alternativa más razonable cuando el control ocurre cerca del dispositivo, ya que evita depender de infraestructura adicional y reduce complejidad operativa.

Sin embargo, el diagrama ya contempla una abstracción mediante ICommunicationAdapter, por lo que resulta técnicamente correcto dejar preparado un segundo adapter para MQTT.

MQTT no es obligatorio en el MVP, pero sí es una evolución lógica si el proyecto más adelante requiere monitoreo remoto, telemetría, notificaciones o administración de múltiples equipos.

4.7 Stores, services y repositories propios

No se prioriza inicialmente una librería adicional de gestión de estado porque el diagrama ya define stores especializados: AppStore, RecipeStore y SettingsStore. Para el tamaño y alcance actual, esta aproximación es suficiente y mantiene el sistema comprensible.

Los services concentran reglas de aplicación y coordinación con la capa de comunicación; los repositories aíslan el acceso a datos locales. Esta separación mejora testeabilidad, mantenibilidad y claridad arquitectónica.

En resumen, la estructura seleccionada privilegia una arquitectura limpia con pocas dependencias obligatorias, alineada con el tamaño real del proyecto.

5. Decisiones de diseño derivadas del diagrama de clases

Gestión de estado central. La presencia de AppStore indica la necesidad de una fuente común de verdad para el estado global de la máquina. Esto evita duplicación de estado entre pantallas y facilita que una acción en Control se refleje correctamente en Tragos o Ajustes.

Patrón Adapter para la comunicación. ICommunicationAdapter, BLEAdapter y MQTTAdapter muestran una decisión correcta de desacoplamiento. Gracias a ello, DeviceService puede depender de una interfaz y no de una implementación concreta. Esta elección reduce impacto de cambios futuros.

Patrón Repository para persistencia. RecipeRepository, SettingsRepository e HistoryRepository encapsulan el acceso a LocalDatabase. Con esto, la app no mezcla SQL o detalles de almacenamiento dentro de las pantallas ni dentro de los stores.

Cola de comandos. CommandQueueService es una decisión importante para escenarios reales: pérdida temporal de conexión, reintentos, ejecución secuencial o confirmación diferida. Este componente aporta robustez a una app IoT.

Modelo de dominio explícito. Clases como Recipe, Ingredient, MachineState, DeviceCommand y PreparationRecord ayudan a mantener un lenguaje de dominio claro y favorecen trazabilidad entre requerimientos funcionales y código.

6. Tecnologías no priorizadas o descartadas en esta etapa

Elemento	Motivo
Flutter	Aunque es una alternativa fuerte para apps multiplataforma, no se eligió porque el equipo ya domina React y el costo de cambio no se justifica para este proyecto.
SQLite dentro del ESP32	No se propone como almacenamiento principal en el microcontrolador. Para el dispositivo embebido es más razonable persistir configuración y estados mediante NVS, mientras la app resuelve la persistencia estructurada con SQLite.
Backend obligatorio desde el inicio	No se considera una dependencia obligatoria para el MVP. Primero conviene validar el flujo local de control; un backend puede incorporarse después para sincronización, métricas o administración remota.

7. Recomendaciones de implementación

1. Organizar el proyecto por features o por dominios funcionales, manteniendo separadas las carpetas de screens, components, stores, services, repositories, adapters y models.
2. Implementar primero BLEAdapter y dejar MQTTAdapter como interfaz preparada, pero sin convertirlo en dependencia crítica del MVP.
3. Mantener a LocalDatabase como punto único de apertura de base y centralizar migraciones de esquema para evitar inconsistencias.
4. Registrar en HistoryRepository los eventos de preparación más importantes para futuras analíticas, soporte técnico y trazabilidad operativa.
5. Definir tipos compartidos para payloads de comandos y estados devueltos por el dispositivo, evitando strings mágicos y estructuras improvisadas.

Recomendación de cierre técnico. Para la primera versión conviene fijar el alcance en React Native + TypeScript + Expo + Expo Router + SQLite + BLE. Esa combinación cubre el MVP con una base ordenada y deja el camino abierto para integrar MQTT o backend cuando el producto necesite más alcance.

8. Conclusión

La selección tecnológica propuesta responde tanto al contexto del equipo como a la estructura del software modelada en el diagrama de clases. React Native, TypeScript, Expo, Expo Router y SQLite forman una base coherente para una app móvil modular, mantenible y orientada a iteración rápida. La arquitectura queda preparada para operar localmente con BLE en el corto plazo y para evolucionar hacia MQTT y servicios remotos en el mediano plazo. En consecuencia, el stack recomendado no solo es técnicamente viable, sino también consistente con la estrategia de implementación más eficiente para el proyecto.

Referencias técnicas consultadas

- React Native Documentation. “Get Started with React Native” y documentación general del framework.
- Expo Documentation. “Introduction to Expo Router”, “Expo Router”, “SQLite” y guías de almacenamiento local.
- TypeScript Handbook. Documentación sobre tipado estático y fundamentos del lenguaje.
- ESP-IDF Programming Guide. Documentación oficial de Bluetooth, ESP-MQTT, NVS y consideraciones de sistema de archivos para ESP32.